

Real-Time Environmental Monitoring with Edge-Deployed ML for Anomaly Detection and Forecasting

Pratik Kumar¹, Thakur Chand Choudhary², Nikunj Goydani³, Vetriveeran Rajamani^{1*}, Poongundran

School of Electronics Engineering (SENSE), Vellore Institute of Technology (VIT), Vellore, Tami

¹pratik.kumar2024a@vitstudent.ac.in, ²thakur.chand2024@vitstudent.ac.in, ³nikunj.goydani

^{1*}vetriecec86@gmail.com, vetriveeran.r@vit.ac.in, ^{2*}poongundran.selvaprabhu@

Abstract

We designed and implemented an IoT-based environmental monitoring device (EMD) that uses an ESP8266 microcontroller and a CD4051 analog multiplexer to provide multiple analog sensors to the single ADC of the ESP8266. It contains multiple sensors including DHT22 (temperature/humidity), MQ-6 (gases), LDR (light level), and soil moisture sensor. It displays readings locally on a JHD162A LCD (I2C) and transmits data via Wi-Fi to a Flask backend. The data is saved in SQLite or Supabase (PostgreSQL) and visualized using a React web application with Plotly charts. The use of an ML pipeline allows for the creation of LSTM models for multi-step predictions of the five environmental parameters based on recorded historical data, along with the use of an Isolation Forest model for detecting anomalies in the data recorded from all connected sensors. The models were trained on data recorded in a laboratory and deployed for five days; these results show that the system is stable and consistently operates with Wi-Fi connectivity, uploading the recorded data to a cloud service (supabase.com) every 10 seconds while maintaining data integrity through the use of an effective anomaly detection algorithm and accurate time-series predictions of sensor data.

Index Terms

ESP8266, CD4051, analog multiplexer, DHT22, MQ-6, LDR, soil moisture, Flask, React, LSTM, Isolation Forest, environmental monitoring, time-series prediction, anomaly detection

I. INTRODUCTION

In the last few years, the importance of environmental monitoring has increased significantly as regards the smart building sector, the precision agriculture industry, industrial safety and laboratory quality assurance. The increased use of low-cost microcontrollers and their ability to connect to networks has made it possible to create devices that can use these microcontrollers to form distributed sensor networks capable of measuring temperature, humidity, air quality, light, and soil conditions in real-time, as well as transmitting that data to cloud or edge analytics platforms. The research in this area covers the integration of hardware, the development of communication protocols, the storage of data, the creation of data visualizations, and how ML can be applied for detection of anomalies and predictive analytics [1], [3].

One of the most significant challenges with low-cost IoT nodes [7], [8] is the lack of ADC resources. Many of the commonly used microcontrollers feature only one or a few ADC channels, and as a result, have a limited number of analog sensors that will be able to connect without using either an external expansion method or having to use multiple devices. Historically, analog multiplexers (MX) have been used in embedded systems to allow multiple analog sources to share the same ADC. However care must be taken with the design of the interface to account for channel selection timing, settling time, and wiring design to ensure that there is no crosstalk and that there are accurate measurements being taken.

The ongoing research of integrating various environmental parameters into a unified pipeline spanning from the acquisition of data through its eventual storage/visualization and the use of ML algorithms to make predictions and detect anomalies continues to be an area of focus in research initiatives, particularly when models are trained and tested exclusively using field/laboratory data rather than with synthetic data sets [6]. In this paper we discuss the design and evaluation of such a system along three axes: the use of a CD4051 multiplexer with an ESP8266 node to gather readings from gas, light, and soil moisture sensors via one ADC plus digital temperature and humidity sensors; the implementation of a back end and web-based dashboard to store and visualize the collected data; and the provision of an ML pipeline with LSTM-based time-series predictions and Isolation Forest-based anomaly detection, trained using only data collected during five days of deployment in the laboratory. This study is intended as a research-oriented treatment of the hardware, software, and ML integration rather than simply being a project report; the focus is on documented and reproducible design decisions, data handling, and evaluation methodologies. The key contributions are a documented interface for connecting a CD4051 to an ESP8266; methods for designing and building an environmental monitoring system; a substantial corpus of temporal environmental data (to our knowledge not currently available elsewhere in the same form); and a structured ML pipeline with multiple algorithms for prediction and anomaly detection using only lab-collected data.

This paper is organized as follows. The second section reviews literature relevant to environmental monitoring in the context of IoT, ADCs and ML applications to sensor data telemetry, and Section three presents the architecture of the system being developed. Section four will discuss how the hardware was implemented using CD4051 technology, and Section five describes the software environment used in conjunction with this system, Section six discusses results obtained by measuring laboratory data, and Section seven concludes with a summary of findings and potential future research directions.

II. LITERATURE REVIEW

DEFINITIONS OF ACRONYMS

TABLE I: Acronyms and abbreviations used in this paper.

Acronym	Definition	Acronym	Definition
ADC	Analog-to-Digital Converter	MAE	Mean Absolute Error
API	Application Programming Interface	MCU	Microcontroller Unit
AWS	Amazon Web Services	ML	Machine Learning
CD4051	8-ch. CMOS analog Mux (part no.)	MQ-6	Gas sensor module (MQ series)
CoAP	Constrained Application Protocol	MQTT	Message Queuing Telemetry Transport
CMOS	Complementary Metal-Oxide-Semiconductor	MUI	Material-UI (React library)
CSV	Comma-Separated Values	MX	Multiplexer
DB	Database	POST	HTTP request method
DHT22	Digital temp./humidity sensor (model)	REST	Representational State Transfer
EMD	Environmental Monitoring Device	RH	Relative Humidity
GPIO	General Purpose Input/Output	RMSE	Root Mean Square Error
HTTP	Hypertext Transfer Protocol	SCL	Serial Clock (I2C line)
I2C	Inter-Integrated Circuit (serial bus)	SDA	Serial Data (I2C line)
IoT	Internet of Things	SENSE	School of Electronics Engineering
JSON	JavaScript Object Notation	SQLite	Lightweight SQL database engine
LDR	Light-Dependent Resistor	VIT	Vellore Institute of Technology
LCD	Liquid Crystal Display	Vite	Front-end build tool
LoRaWAN	Long Range Wide Area Network	Wi-Fi	Wireless Fidelity
LPG	Liquefied Petroleum Gas	Zigbee	Low-power wireless mesh protocol
LSTM	Long Short-Term Memory		

This section surveys related work in IoT-based environmental monitoring, hardware constraints and ADC expansion, cloud backends and visualization, and ML for sensor data analysis. The aim is to position the present system within existing research and to identify the gaps that this work addresses.

A. IoT environmental monitoring and low-cost nodes

The use of low-cost microcontrollers for environmental sensing has been widely reported. Kumar and Sharma [1] integrated such nodes with MQTT and cloud platforms for air quality monitoring, demonstrating the viability of cost-effective sensor-to-cloud pipelines. Gupta and Das [2] developed a NodeMCU-based climate control system with Firebase for real-time data exchange, though their work noted limitations in offline storage and local buffering. ESP8266-based deployments have received particular attention owing to built-in Wi-Fi and a large developer ecosystem. Rao and Mehta [7] used the ESP8266 for environmental data acquisition and analytics, while Singh and Reddy [8] focused on gas detection and alerting with MQ-series sensors on the same platform; both studies highlighted challenges such as calibration drift and the limited number of analog inputs on the device. Beyond single-node designs, wireless sensor networks have been proposed for larger-scale monitoring: Wang et al. [10] employed Zigbee for environmental monitoring, and Ali et al. [11] presented a LoRaWAN-based scalable system, trading higher coverage and power efficiency for increased protocol and infrastructure complexity. These works establish a clear trend toward low-cost, connected environmental sensing but often assume either multiple ADC channels or a single sensor type per node, leaving the problem of multi-parameter analog acquisition on a single-ADC device less fully addressed in the literature.

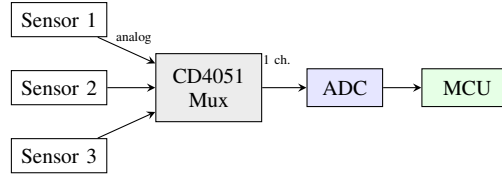
B. ADC expansion and analog multiplexing

Most microcontrollers have one or two channels of ADC circuitry which limits the number of analog sensors that can connect directly to them without the need of any external device. The CD4051 and CD4052 multiplexers allow an analog signal from multiple sources to be sampled onto one ADC; thus, an interface (mounting board) between an ADC and multiple analog signals is required. For embedded applications such as instrumentation, design requirements are defined for signal ground placement, switching time and settling time for channels selected. Scalable and reconfigurable sensor network systems have previously been described in the literature; particularly prior work on modular sensor fusion and adaptable environmental monitoring services [21] has no specific details on interfacing or timing between the CD4051 and ESP8266; however, in the present situation, the ESP8266 provides only one 10-bit A/D converter (A0); therefore, any multi-channel (analog) design using an ESP8266 must use an

TABLE II: References and research summary: IoT environmental monitoring and low-cost nodes.

Reference	Research Summary
Kumar and Sharma [1]	Presented a cloud-based air quality monitoring system using low-cost IoT sensors and MQTT, demonstrating cost-effective sensor-to-cloud pipelines. The work showed that such pipelines are viable for continuous environmental monitoring but did not address multi-channel analog acquisition on resource-constrained nodes.
Gupta and Das [2]	Developed an IoT-enabled climate control system using NodeMCU with Firebase for real-time data exchange. The study highlighted limitations in offline storage and local buffering, and noted the need for robust connectivity and data persistence in low-cost deployments.
Rao and Mehta [7]	Used the ESP8266 for environmental data acquisition and analytics, establishing its suitability for connected sensing. The work did not focus on extending the single ADC to multiple analog sensors via multiplexing.
Singh and Reddy [8]	Implemented gas detection and alerting with MQ-series sensors on the ESP8266 platform. They reported issues such as calibration drift and the limited number of analog input channels on the device, motivating the need for ADC expansion.
Wang et al. [10]	Deployed a Zigbee-based wireless sensor network for environmental monitoring, enabling larger-area coverage. The approach uses a different communication stack than Wi-Fi and does not address single-ADC multi-sensor interfacing.
Ali et al. [11]	Proposed a LoRaWAN-based scalable environmental monitoring system, trading increased coverage and power efficiency for greater protocol and infrastructure complexity. The work is complementary to single-node designs that require simple backhaul.

external multiplexing device. This document reports the connection interface pin-outs, settling delay, and software sequences that are used when measuring various sensors that use an ESP8266 A/D channel to time multiplex data from those sensors. The concept of sharing one ADC via an analog multiplexer is illustrated in Fig. 1.

**Fig. 1:** Analog multiplexing: multiple sensors share a single ADC via a multiplexer (e.g., CD4051).**TABLE III:** References and research summary: ADC expansion and analog multiplexing.

Reference	Research Summary
Kumar et al. [21]	Addressed modular sensor fusion and adaptable environmental monitoring frameworks, enabling customizable deployment. The work does not provide specific details on CD4051–ESP8266 interfacing, channel-select timing, or settling delays, which are needed for reliable multi-channel analog acquisition on a single ADC.

C. Laboratory, industrial, and agricultural monitoring

According to the studies [6], [15], multiple environmental parameters used for various applications require the use of configurable sampling, storage, and alerting systems. Configurable and modular systems have been reported for environmental monitoring in laboratory settings that take into account multiple different types of physical quantities (i.e., temperature/humidity), with the flexibility of how/when data is logged. Similarly, in manufacturing and industrial settings, studies [16], [17] demonstrate the requirement for real-time, adaptive systems to help ensure chemical reactor safety and to control the manufacture of chemical/processes, where the monitoring of gas/temperature/pressure is standard [20]. For agricultural and controlled environment growth applications, IoT-based systems have been reported to help with the climate control of the environment and with modular sensor networks that monitor components of the growth process (i.e., soil moisture/light/temp/humidity) which provide additional resources for increasing crop yield [18], [19]. Therefore, the emphasis of these publications supports a consistent requirement for multi-parameter sensing (i.e., temperature, humidity, gas, light, soil moisture) in laboratory, industrial and agricultural applications and for back-end systems that allow for data storage, visualization, and potential analysis.

D. Cloud backends, storage, and visualization

When designing a backend for IoT environmental data, there are three important choices: local vs. cloud storage, REST vs. MQTT/CoAP, and visualization dashboard types. Lightweight relational storage (like SQLite [13]) works

TABLE IV: References and research summary: laboratory, industrial, and agricultural monitoring.

Reference	Research Summary
Chen and Zhang [6]	Presented configurable IoT sensor networks for precision laboratory environmental monitoring, emphasizing multiple physical quantities and flexible data logging. The design supports the need for multi-parameter sensing in research and quality-assurance settings.
Anderson et al. [15]	Described modular environmental monitoring systems for research laboratories, including design and implementation aspects. The work reinforces the requirement for configurable sampling, storage, and alerting across lab applications.
Park and Kim [16]	Addressed real-time customizable monitoring systems for chemical reactor safety, where gas, temperature, and pressure monitoring are critical. The study illustrates the need for adaptive and reliable sensing in industrial environments.
Rodriguez et al. [17]	Developed adaptive IoT platforms for industrial process monitoring and control, linking sensor data to decision support. The work aligns with the need for back-end systems that support storage, visualization, and analytics.
Thompson and Lee [20]	Discussed scalable and reconfigurable IoT architectures for industrial applications, enabling flexible deployment of sensor networks. The findings support the use of modular designs for multi-parameter environmental monitoring.
Li and Wang [18]	Reported IoT-based climate control systems for smart greenhouses, incorporating soil moisture, light, temperature, and humidity. The work demonstrates the relevance of multi-parameter sensing in agricultural and controlled-environment applications.
Martinez et al. [19]	Presented modular sensor networks for precision agriculture in controlled environments, improving resource use and yield. The study underscores the demand for integrated sensing of temperature, humidity, gas, light, and soil moisture.

great for either single node/small-scale or embedded/edge servers, while cloud platforms (like AWS IoT [14]) allow for large-scale analytics and multi-tenancy; however, at the cost of operating complexity. RESTful APIs are often preferred for simple push and query scenarios and easily work with web dashboards. Streamlit [12] is an example of a rapidly developed data science or IoT dashboard, while modern web frameworks (e.g., React + chart libraries) enable interactive, real-time displays of sensor time series and exported data. The system developed in this paper uses a Flask-based REST API with either SQLite or Supabase (PostgreSQL) for its backend and a React-based dashboard for visualization, resulting in balance among simplicity, portability, and the ability to show multiple parameters and/or machine-learning based predictions within the same interface.

TABLE V: References and research summary: cloud backends, storage, and visualization.

Reference	Research Summary
Hipp [13]	Described SQLite as a lightweight relational database engine well suited to embedded and small-scale systems. The work supports the choice of local or edge storage for IoT environmental data when simplicity and portability are priorities.
Patel and Jain [14]	Demonstrated scalable IoT analytics using the AWS cloud platform, enabling large-scale analytics and multi-tenancy. The study illustrates the trade-off between operational complexity and scalability when moving from local to cloud backends.
Carns [12]	Introduced Streamlit for rapid development of data science and IoT dashboards. The tool exemplifies the trend toward interactive, real-time visualization of sensor time series and exported data in modern web frameworks.

E. ML for sensor data

Increased use of ML has taken place in environmental and industrial sensor data for purposes such as anomaly detection, forecasting and decision support. [3] reviewed the literature regarding the use of ML to detect anomalous values within IoT sensor data and demonstrated that many of the ML techniques described can be useful in identifying anomalous values that are not identified using fixed thresholds. LSTMs are frequently used for time series prediction using recurrent architectures when sufficient sequential data exists; research and industry applications have occurred with temperature, humidity and other environmental time series. A limitation of many of the systems in the literature have been that they are trained or evaluated using synthetic or augmented data; the preferred method of training and evaluating a predictive model for deployment in an actual laboratory or facility is by using models that are trained and validated using only collected data since this will provide predictions and/or anomaly flags that more accurately reflect actual operating conditions. Isolation Forest and related algorithms are effective solutions for anomaly detection in cases where few labeled anomalies exist because these algorithms will learn a representation of “normal” from the recorded distribution of values [3]. In the current research study, the researcher trained LSTM models to perform multi-step predictions for each of the five parameters that were recorded and used the Isolation

Forest algorithm to detect anomalies and utilized collected data for both the training and evaluation process. Fig. 2 summarizes the ML pipeline themes discussed in the literature.

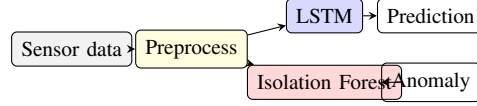


Fig. 2: ML for sensor data: time-series prediction (LSTM) and anomaly detection (Isolation Forest).

TABLE VI: References and research summary: ML for sensor data.

Reference	Research Summary
Verma and Singh [3]	Reviewed and demonstrated ML-based anomaly detection in IoT sensor data, showing that such methods can identify deviations that fixed thresholds miss. The work discusses Isolation Forest and related algorithms that learn a notion of “normal” from recorded distributions, and supports training and evaluation on collected rather than synthetic data.

F. Positioning of the proposed system

This system plays an important role in many ways [6], [21]. First, it gives you a physical and repeatable way to connect up your CD4051 to an ESP8266 for doing multi-channel analogue acquisition that includes everything from channel select timing and wiring to making it so each ADC pin can be used for a gas sensor, a light sensor, a soil moisture sensor, and a digital temperature and humidity sensor at the same time. Secondly, it allows you to integrate five different environmental parameters end-to-end (from start to finish) with a Flask backend, storing all of your data in either an SQLite database or a Supabase database [13], and visualising and exporting that data through a React-based dashboard. Lastly, it also implements an ML pipeline (LSTM for predicting future values; Isolation Forests for identifying anomalies) that uses only laboratory data collected during a 5-day deployment period [3]. There were no synthetic or generated data used when training and evaluating the ML models; therefore, we have strictly adhered to using actual laboratory data only for all aspects described above. These topics/sections will be described in greater detail in later sections.

TABLE VII: References and research summary: positioning of the proposed system.

Reference	Research Summary
Kumar et al. [21]	Provided modular sensor fusion and customizable environmental monitoring frameworks, which the present work extends with a concrete CD4051–ESP8266 interface and lab-data-only ML pipeline.
Chen and Zhang [6]	Addressed configurable IoT sensor networks for precision laboratory environmental monitoring, aligning with the present system’s focus on multi-parameter sensing and reproducible design for lab deployment.
Hipp [13]	Established SQLite as a lightweight database suitable for local or edge storage; the present backend supports SQLite and Supabase for flexibility between local and cloud deployment.
Verma and Singh [3]	Supported the use of ML for anomaly detection and the importance of training and evaluation on collected (non-synthetic) data, which the present LSTM and Isolation Forest pipeline follows.

Table VIII compares the proposed system with other recent works in IoT-based environmental monitoring and related areas. Table IX summarizes the application scenarios and contributions of key references.

III. SYSTEM OVERVIEW

The architecture consists of four main elements: an ESP8266-based sensor node with CD4051 multiplexed analog inputs; a Flask backend with SQLite or Supabase storage; a React web dashboard for visualization; and an ML pipeline that trains on recorded sensor data for prediction and anomaly detection. Fig. 3 illustrates the data flow from the sensors through to the web dashboard and the ML pipeline.

A. Sensor node

The node is built around the ESP8266MOD. The chip has one ADC input (A0); the CD4051 multiplexer routes one of three analog signals (MQ-6, LDR, soil moisture) to A0 under software control. The node interfaces with:

- **DHT22:** Digital temperature and relative humidity; data pin at GPIO2 (D4) with 10 kΩ pull-up. Sampled every 10 s.

TABLE VIII: Comparative analysis of the proposed system with other recent works in IoT environmental monitoring and ML for sensor data.

Year	Ref	Contribution	Scope					
			IoT env.	ADC/ Mux	Cloud/ back.	ML pred.	Anom. det.	Lab data
2019	[1]	Low-cost IoT, MQTT, cloud air quality	✓	✗	✓	✗	✗	✗
2021	[2]	NodeMCU climate, Firebase; of-line limits	✓	✗	✓	✗	✗	✗
2022	[7]	ESP8266 env. data acquisition	✓	✗	✓	✗	✗	✗
2023	[8]	Gas detection, MQ sensors, ESP8266	✓	✗	✓	✗	✗	✗
2018	[10]	Zigbee WSN environmental monitoring	✓	✗	✓	✗	✗	✗
2020	[11]	LoRaWAN scalable env. monitoring	✓	✗	✓	✗	✗	✗
2022	[3]	ML anomaly detection in IoT data	✓	✗	✓	✓	✓	✗
2022	[21]	Modular sensor fusion; no CD4051 detail	✓	✗	✓	✗	✗	✗
–	Proposed	CD4051–ESP8266, 5 params, Flask/React, LSTM+Isolation Forest, lab data only	✓	✓	✓	✓	✓	✓

TABLE IX: Application of IoT and ML in environmental monitoring and related sectors.

Author	Application Scenario	Summary
Kumar and Sharma [1]	Air quality monitoring	Cloud-based system using low-cost IoT sensors and MQTT for sensor-to-cloud pipelines.
Gupta and Das [2]	Climate control	NodeMCU-based climate control with Firebase for real-time data; limitations in offline storage and local buffering.
Rao and Mehta [7]	Environmental analytics	Data acquisition through ESP8266 for environmental data and analytics.
Singh and Reddy [8]	Gas detection	Gas detection and alerting using MQ-series sensors and ESP8266; calibration drift and limited analog channels noted.
Wang et al. [10]	Environmental monitoring	Wireless sensor network for environmental monitoring based on Zigbee.
Ali et al. [11]	Scalable monitoring	LoRaWAN-based scalable environmental monitoring; trade-off between coverage, power efficiency and protocol complexity.
Verma and Singh [3]	Anomaly detection	ML-based anomaly detection in IoT sensor data; ML techniques to identify deviations that fixed thresholds miss.
Chen and Zhang [6]	Laboratory monitoring	Configurable IoT sensor networks for precision laboratory environmental monitoring.
Li and Wang [18]	Smart greenhouse	IoT-based climate control for smart greenhouses; soil moisture, light, temperature, humidity.
Martinez et al. [19]	Precision agriculture	Modular sensor networks for precision agriculture in controlled environments.

- **MQ-6:** Gas sensor (e.g., LPG); 5 V supply, analog output connected to CD4051 channel 0. Analog value digitized via A0 after channel selection.
- **LDR:** Light-dependent resistor module; analog output to CD4051 channel 1. Higher ADC value corresponds to higher light level.
- **Soil moisture:** Analog module output to CD4051 channel 2; higher reading indicates wetter soil (module-dependent).
- **CD4051:** 8-channel analog multiplexer; select pins S0, S1, S2 driven by GPIO12 (D6), GPIO13 (D7), GPIO14 (D5); common output Z to ESP8266 A0. Channels 0, 1, 2 used; 3–7 reserved for expansion.
- **JHD162A LCD (I2C):** SDA on GPIO4 (D2), SCL on GPIO5 (D1). Cycles through temperature/humidity, gas, light/soil, and status screens.
- **HW-131:** DC-DC power module; 5 V output for ESP8266 (VIN), MQ-6, and LCD; 3.3 V derived where required for DHT22, LDR, soil module, and CD4051.

Firmware selects each CD4051 channel in turn, applies a short settling delay (1–2 ms), then reads A0. All five parameters are sent in one JSON payload via HTTP POST to the backend every 10 s. On Wi-Fi or sensor failure, status is shown on the LCD and transmission is retried.

B. Backend API

The backend is implemented in Flask and exposes REST endpoints:

- `/api/sensor-data`: POST from ESP8266; stores timestamped temperature, humidity, gas_analog, light_level, soil_moisture in SQLite or Supabase.

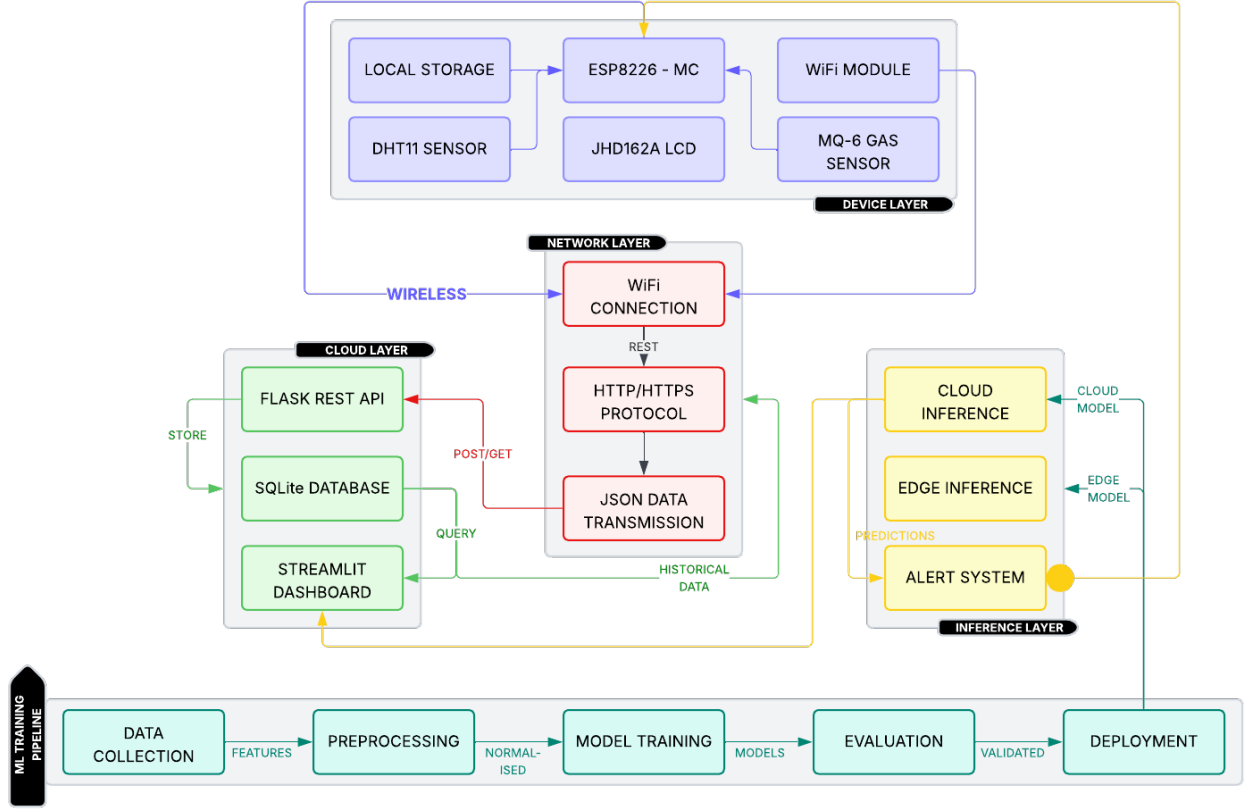


Fig. 3: System architecture: sensor node (ESP8266 + CD4051 + sensors), Flask API, database, React dashboard, and ML pipeline.

- `/api/latest`: Returns the most recent reading (all five parameters) for the dashboard.
- `/api/stats`: Aggregated statistics (min, max, average) over configurable time ranges for all parameters.
- `/api/health`: Backend and database health.
- `/predict/temperature`, `/predict/humidity`, `/predict/gas`, `/predict/light`, `/predict/soil`: Prediction endpoints returning LSTM-based forecasts for the requested parameter (e.g., next 1–168 h).

Input validation and error handling are applied; the same schema supports local SQLite and Supabase (PostgreSQL) for deployment flexibility.

C. Data dashboard

The dashboard is a React (Vite) single-page application with Material-UI and Plotly. It provides:

- **Live and historical charts:** All five parameters visualized with auto-refresh; time-range selection and Plotly zoom/pan.
- **Predictions view:** Single chart showing LSTM predictions for available parameters over a user-selected horizon (e.g., next 24 h), with values normalized for comparison and original units on hover.
- **CSV export:** Recorded data export for offline analysis.
- **System status:** Last update time, backend health, connectivity indicators.

No synthetic or generated data is used; all displayed and exported series are from recorded sensor data and model predictions trained on that data.

D. ML pipeline

- **Data:** Time-series of temperature, humidity, gas_analog, light_level, and soil_moisture with timestamps are loaded from the database. No synthetic or generated series are added.
- **Preprocessing:** Rolling means and change features for all five parameters; optional time-of-day/day-of-week; normalization (e.g., MinMaxScaler). Missing values are filled (e.g., column mean) for robust training.

- **Training: LSTM** models per target parameter (five models), trained on sequences of past observations to predict future values (multi-step up to 168 h); **Isolation Forest** for anomaly detection on the same recorded feature set.
- **Evaluation:** Hold-out or time-based split on recorded data; metrics such as MAE/RMSE for regression and precision/recall for anomaly detection.
- **Deployment:** Trained LSTM and scalars are loaded by the Flask backend; prediction endpoints run cloud-side inference. Anomaly detection can be run on incoming or historical recorded data.

All models are trained exclusively on lab-recorded data; the pipeline does not use generated or synthetic datasets.

IV. HARDWARE IMPLEMENTATION

The hardware is organized around the ESP8266's single ADC (A0). The CD4051 (CMOS 8-channel analog multiplexer/demultiplexer) allows three analog sensors to share A0; the DHT22 uses a digital pin. Key components:

- **ESP8266MOD:** A low-cost, Wi-Fi-enabled system-on-chip microcontroller with an 80 MHz Xtensa core and built-in 802.11 b/g/n connectivity. In this design it provides the single 10-bit ADC (A0), drives the CD4051 select lines, polls the DHT22, controls the LCD over I2C, and runs the HTTP client. Supply: 3.3 V logic; VIN from HW-131 5 V via onboard regulator.
- **DHT22:** A digital temperature and relative-humidity sensor using a capacitive humidity element and a thermistor, with typical accuracies of $\pm 0.5^\circ\text{C}$ and $\pm 2\%$ RH. We connect its data pin to GPIO2 (D4) with a 10 k Ω pull-up to 3.3 V and sample it once per cycle for temperature and humidity.
- **MQ-6:** A semiconductor gas sensor whose conductivity changes in the presence of combustible gases (e.g., LPG, methane). It requires a 5 V heater supply and outputs an analog voltage proportional to gas concentration. We feed its analog output into CD4051 channel 0; readings are digitized via A0 after selecting that channel (warm-up time required before stable readings).
- **LDR module:** A light-dependent resistor whose resistance decreases with increasing ambient light. The module produces an analog voltage representing light level. We connect its analog output to CD4051 channel 1 and read it via the shared A0 after channel selection.
- **Soil moisture module:** An analog sensor that measures soil moisture by detecting changes in electrical conductivity between two probes. Higher moisture gives higher conductivity and typically a higher output voltage. We connect its analog output to CD4051 channel 2 for digitization through the same A0.
- **CD4051:** A CMOS 8-channel analog multiplexer/demultiplexer IC that routes one of eight analog inputs to a common output under digital control. We use it so that three analog sensors (gas, light, soil) share the ESP8266's single ADC. VDD/VCC to 3.3 V, VSS/VEE/INH to GND. Address inputs S0 (A) $\rightarrow$ GPIO12, S1 (B) $\rightarrow$ GPIO13, S2 (C) $\rightarrow$ GPIO14; common pin Z $\rightarrow$ A0. Y0 $\rightarrow$ MQ-6, Y1 $\rightarrow$ LDR, Y2 $\rightarrow$ soil. A 1–2 ms settling delay is applied after each channel change before `analogRead(A0)`.
- **JHD162A LCD (I2C):** A 16 $\times$ 2 character liquid-crystal display with an I2C backpack that reduces wiring to four lines (VCC, GND, SDA, SCL). We use it for local display of temperature, humidity, gas level, light/soil values, and Wi-Fi status. SDA $\rightarrow$ GPIO4 (D2), SCL $\rightarrow$ GPIO5 (D1).
- **HW-131:** A DC-DC buck (step-down) module that accepts a wide input range (typically 7–28 V) and provides a regulated 5 V output. We use it to power the ESP8266 (VIN), MQ-6 heater, and LCD; 3.3 V is derived where needed for DHT22, LDR, soil module, and CD4051.

The LCD cycles through temperature and humidity, gas level with bar, light and soil moisture, and sensor and Wi-Fi status. Firmware samples DHT22 once per cycle, then selects CD4051 channels 0, 1, 2 in sequence with a short delay and reads A0 each time, then sends one JSON payload to the backend every 10 s. The CD4051 leaves channels 3–7 available for future analog sensors without adding another MCU or ADC.

V. SOFTWARE ARCHITECTURE

The stack has four main parts: embedded firmware, Flask backend, React dashboard, and Python ML pipeline.

- **Embedded Firmware (Arduino C++):** Runs on ESP8266. Initializes DHT22 (GPIO2), CD4051 select pins (GPIO12/13/14), I2C LCD (GPIO4/5), and Wi-Fi. Main loop: read DHT22; for each CD4051 channel 0–2, set S0/S1/S2, delay 1–2 ms, read A0; format JSON with temperature, humidity, gas_analog, light_level, soil_moisture; update LCD; POST to `/api/sensor-data`. Retries and LCD messages on failure.

- **Backend (Flask):** Python; SQLite locally or Supabase (PostgreSQL) for production. Routes: sensor-data POST (validate and insert), latest, stats, health, and per-parameter prediction endpoints. Prediction module loads saved LSTM models and MinMax scalers, runs inference, returns time series (e.g., next 1–168 h). All training data is from recorded sensor readings; no synthetic data is injected.
- **Dashboard (React + Vite + MUI + Plotly):** Single-page app calling backend `/api/latest`, stats, and prediction APIs. Real-time and historical charts for all five parameters; a combined prediction chart with normalized traces and original units on hover; CSV export of recorded data; health and last-update indicators.
- **ML pipeline (Python):** Preprocessing: load recorded time-series from DB, build rolling and change features for all five parameters, scale (e.g., MinMax). Train one LSTM per target (temperature, humidity, gas_analog, light_level, soil_moisture) and save model + scalers; train Isolation Forest on the same feature set for anomaly detection. Training uses only recorded lab data (e.g., five-day deployment). Inference is invoked by the Flask prediction endpoints.

A key algorithm that distinguishes this design is the firmware routine that drives the CD4051 multiplexer so that a single ADC pin on the ESP8266 can sample multiple analog sensors reliably. For each channel, the select pins S0/S1/S2 are set to the channel index, a short settling delay (1–2 ms) is applied to avoid crosstalk, and then the shared analog input is read (Listing 1).

Listing 1: CD4051 channel selection and ADC read loop: one ADC serves gas, light, and soil moisture via multiplexer with settle delay.

```

1 // S0,S1,S2 = GPIO12,13,14; shared analog = A0
2 const int S0=12, S1=13, S2=14;
3 int gas_analog, light_level, soil_moisture;
4 for (int ch = 0; ch < 3; ch++) {
5     digitalWrite(S0, ch & 1);
6     digitalWrite(S1, (ch>>1) & 1);
7     digitalWrite(S2, (ch>>2) & 1);
8     delay(2); // 1-2 ms settling
9     int raw = analogRead(A0);
10    if (ch == 0) gas_analog = raw;
11    else if (ch == 1) light_level = raw;
12    else soil_moisture = raw;
13 }

```

End-to-end, data flows from hardware (DHT22 + CD4051 channels) to backend storage, then to the React UI and ML models; all training and evaluation use recorded data only.

VI. RESULTS AND DISCUSSION

The system was evaluated using data recorded in a laboratory over five consecutive days. Sensor readings were sampled every 10s and transmitted to the backend; the recorded dataset was used for storage, visualization, and ML training. No synthetic or generated data was used. Findings are summarized below.

- **Wi-Fi and upload:** The ESP8266 had good connectivity for the entire duration of testing and all data collected during testing had a consistent time interval of ten seconds. The rate of success of HTTP POST requests was very high and if a POST request failed it was retried. No duplicate timestamps or values were stored.
- **Sensor data:** The sensor data was collected continuously and consisted of five metrics, namely temperature, humidity, gas_analog, light_level, and soil_moisture; DHT22 does give an accurate and stable reading in temperature and humidity; The analog readings for gas, light and soil moisture were accurately produced with consistency through the use of CD4051 channel switching with a 1 to 2 ms delay, and thus all data collected was used directly for training and evaluation of the system.
- **Backend and interface:** The Flask-based APIs consumed and delivered the collected data, with React being used to present live and historical records of all five parameters and the ability to both predict and export the results. The aggregation and predictions endpoints responded with acceptable latency times to support interactivity.
- **ML using the collected data:** LSTM models were trained against the five target datasets that contained five days of collected data. A statistical isolation forest model was also trained against the same set of features to identify anomalies. Model evaluation was done using some of the collected data that had not been used in

the training data set (e.g., based upon time). Anomaly detection successfully identified significant deviations, such as gas or temperature spikes, from the collected data; the LSTM predictions accurately matched the most recent short term trends in the collected dataset. No synthetic or generated data were used in the training or evaluation of either ML model.

- **CD4051 and ADC testing:** No channel-to-channel bleeding or crosstalk was observed while recording the collected analog channels. Sequential S0/S1/S2 signals and settle delays were sufficient to provide reliable multi-channel recording.

Fig. 6 shows the React dashboard with real-time sensor data and analytics derived from the recorded dataset. Predictions and anomaly indicators are produced by models trained exclusively on this recorded lab data. The five-day recorded deployment provided sufficient history for LSTM sequence learning and for Isolation Forest to learn normal baselines and flag anomalies in later recorded segments.

Overall, the EMD delivered stable multi-parameter recording via the CD4051–ESP8266 interface, reliable backend storage and API, and ML-based prediction and anomaly detection trained and evaluated only on recorded sensor data.

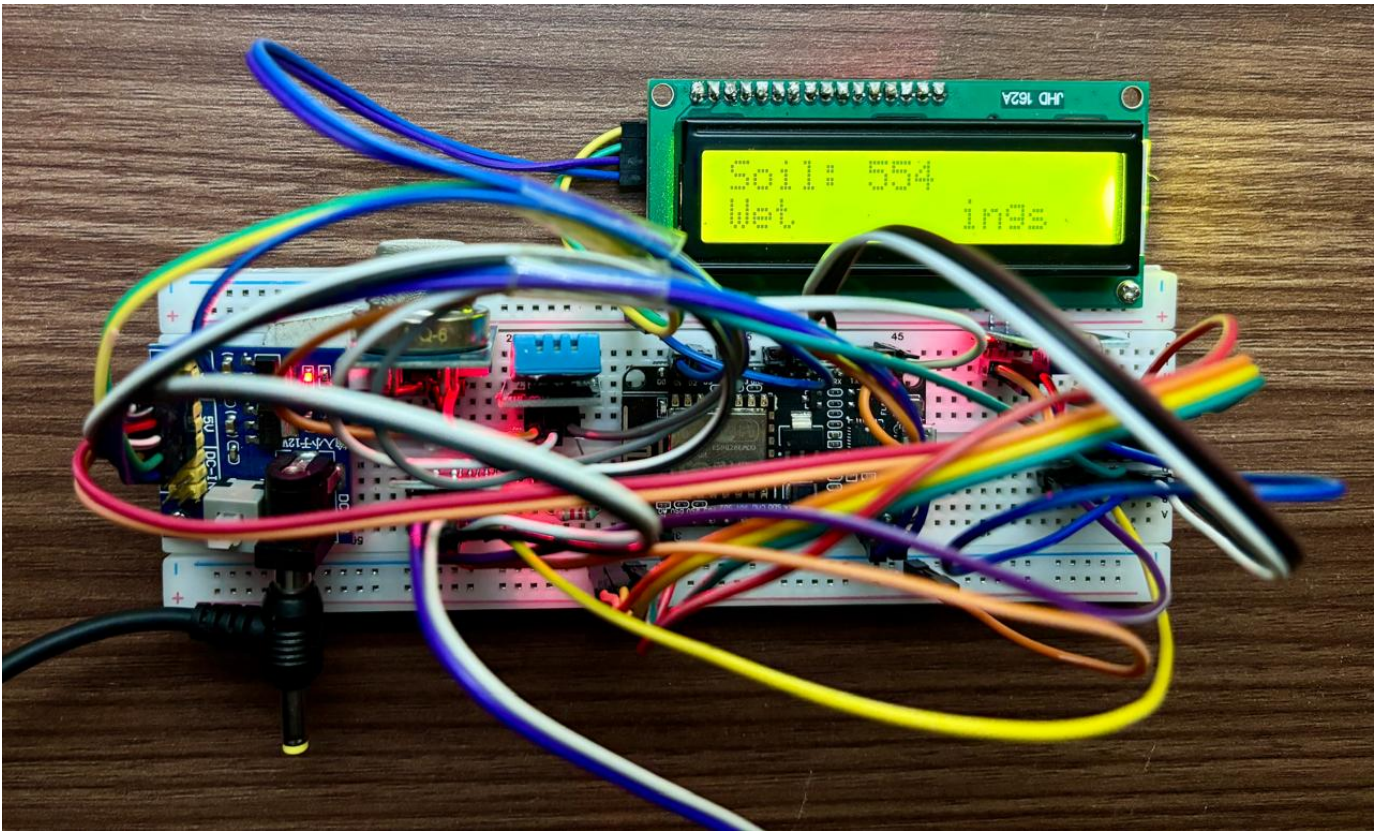


Fig. 4: Hardware setup (breadboard and power).

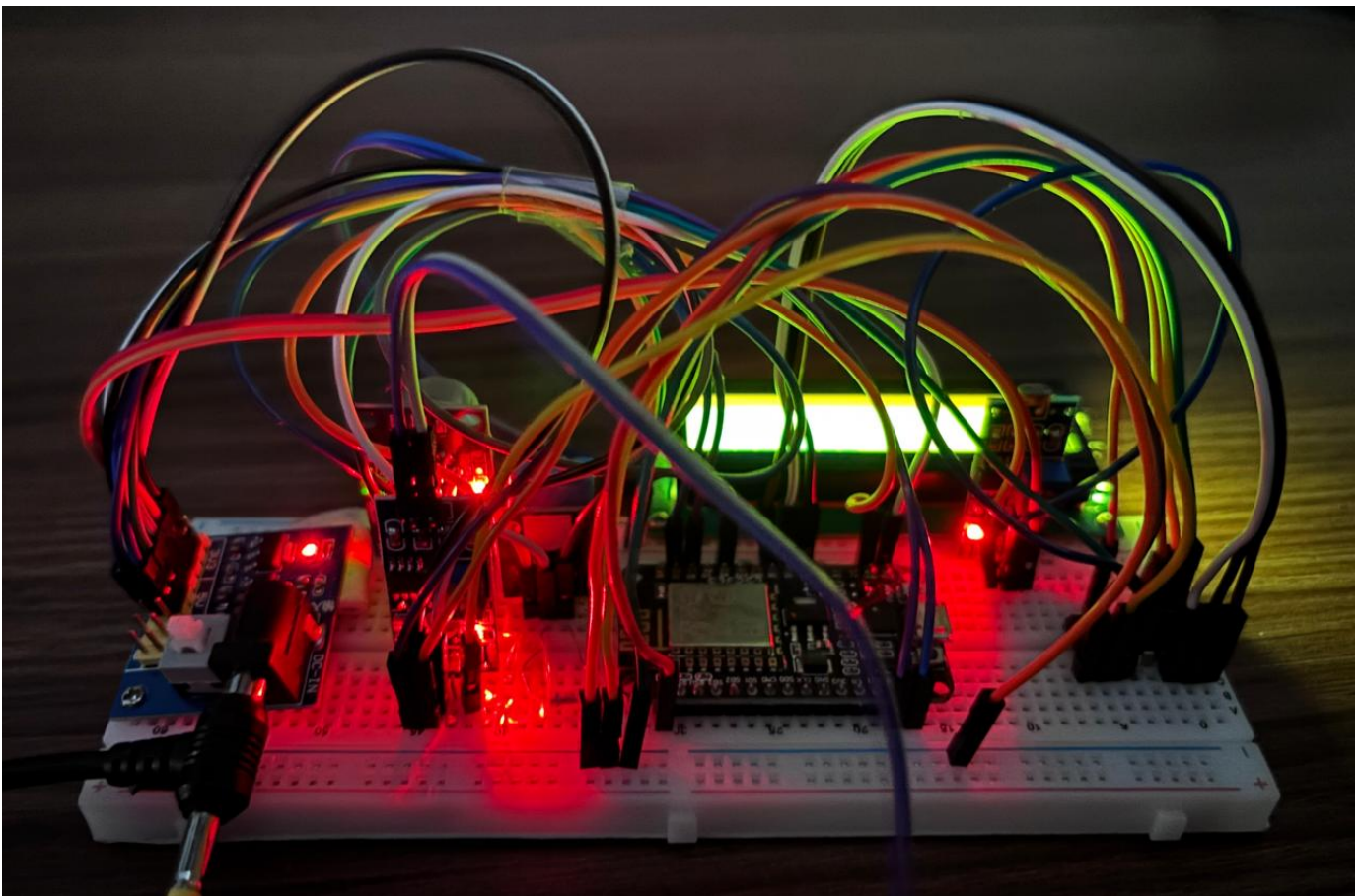


Fig. 5: Hardware setup (LCD showing soil moisture and sensor readings).

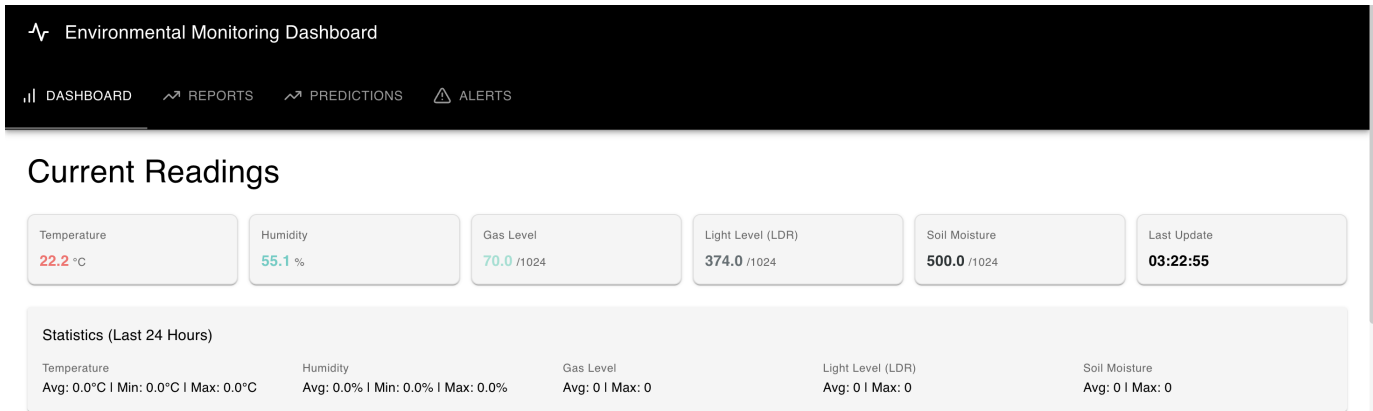


Fig. 6: React dashboard showing current sensor readings, aggregated statistics, and real-time time-series charts for all five environmental parameters (temperature, humidity, gas, light, soil moisture).

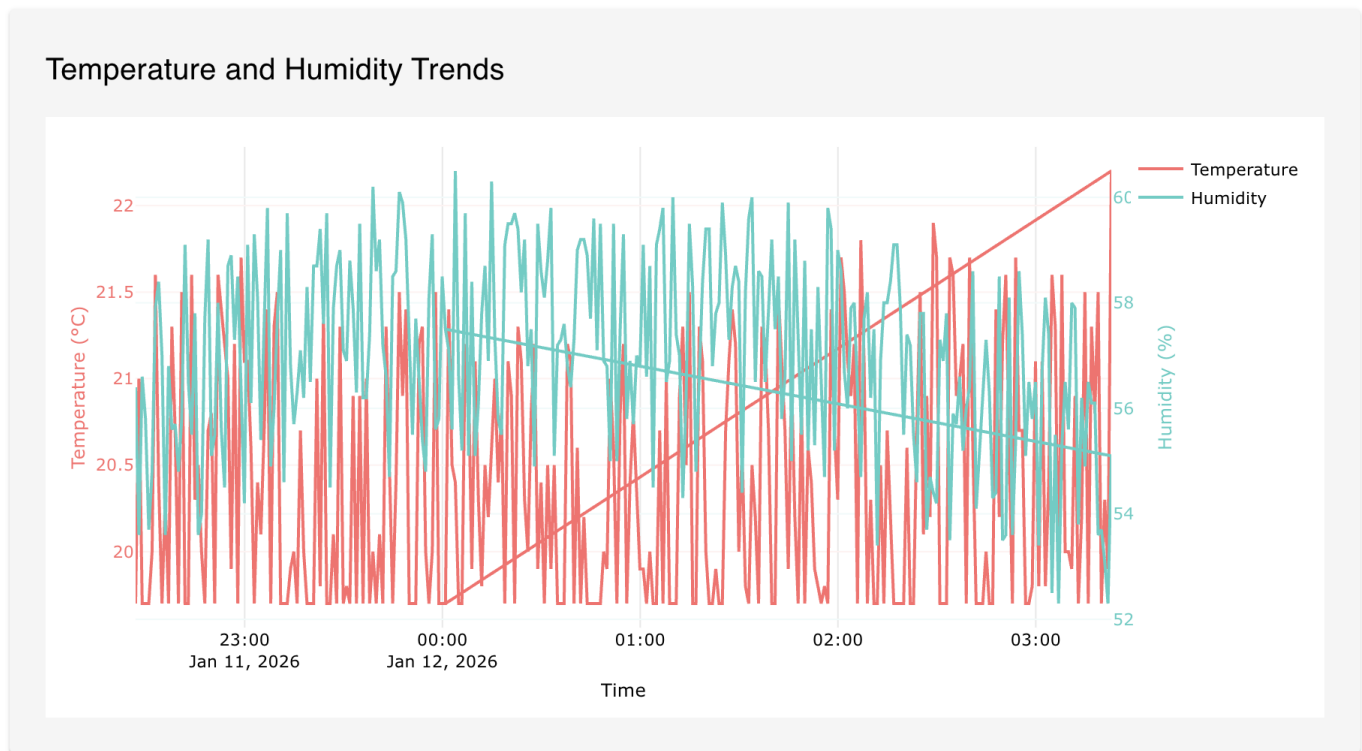


Fig. 7: Temperature and humidity time-series trends over the five-day lab deployment. Data recorded by the DHT22 sensor and displayed via the React dashboard.

Gas Level Trends

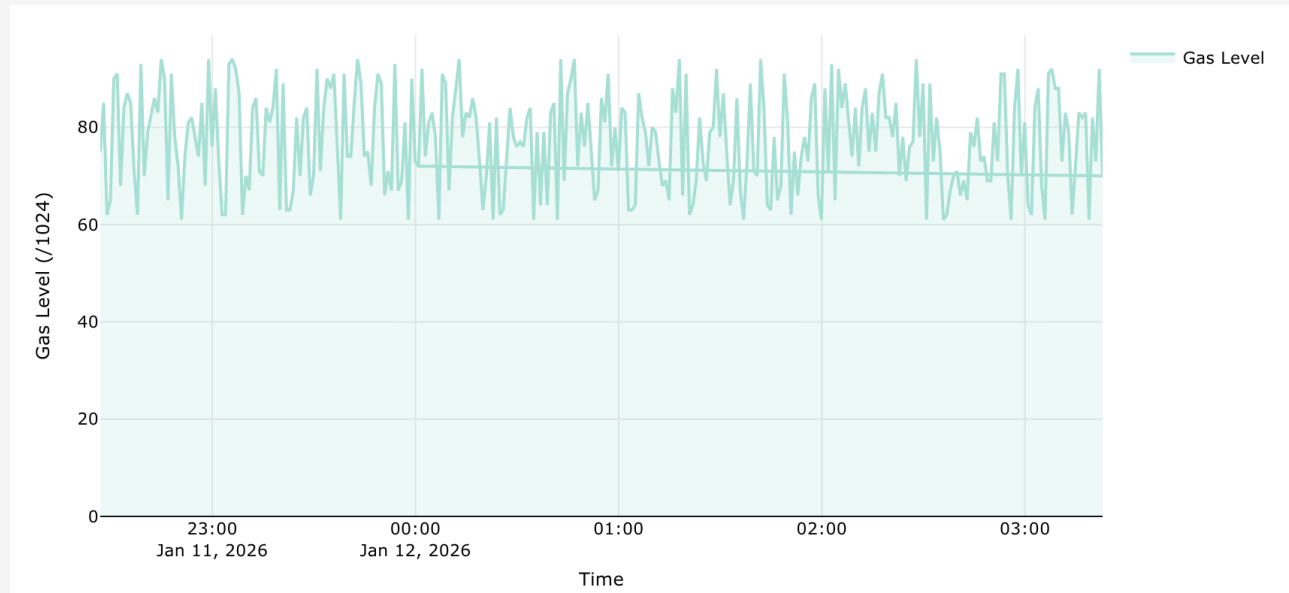


Fig. 8: Gas level (MQ-6 analog reading) time-series trends over the five-day lab deployment. Values digitized via CD4051 channel 0 and the ESP8266 ADC.

LDR & Soil Moisture Trends

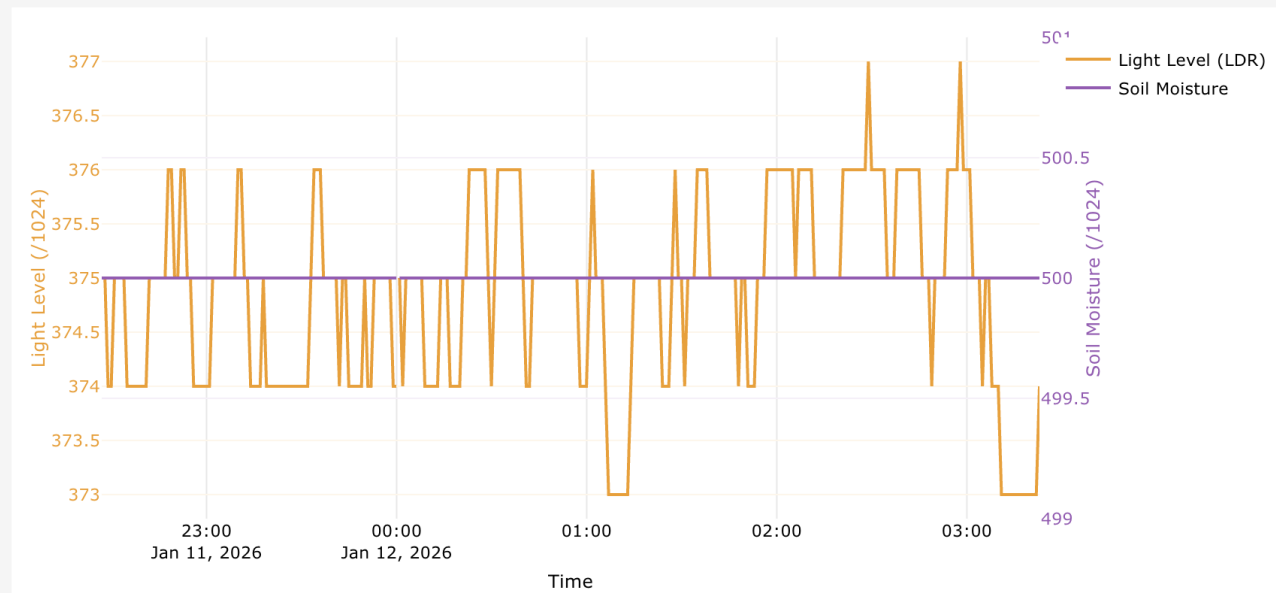


Fig. 9: Light level (LDR) and soil moisture analog readings over the five-day lab deployment. Both channels multiplexed via CD4051 to the single ESP8266 ADC.

Predictive Analytics

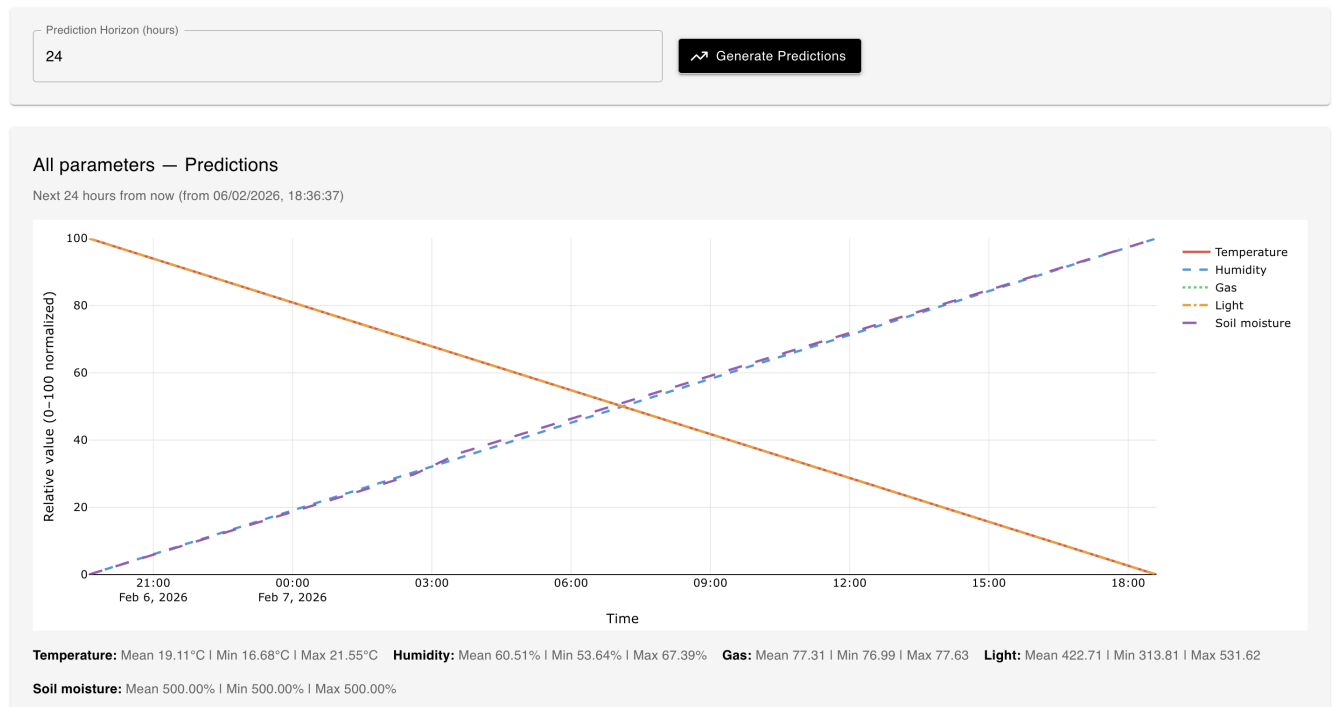


Fig. 10: Multi-step predictions for all five parameters generated by the LSTM models trained on recorded lab data. Normalized traces allow comparison across parameters on a single chart.

VII. CONCLUSION

This paper presented an environmental monitoring device (EMD) that uses an ESP8266 and a CD4051 analog multiplexer to acquire five parameters—temperature, humidity, gas (MQ-6), light (LDR), and soil moisture—using a single ADC. A Flask backend (SQLite or Supabase) stores the data and a React dashboard with Plotly provides visualization. An ML pipeline comprising LSTM-based prediction and Isolation Forest for anomaly detection was trained and evaluated solely on five days of recorded lab data; no synthetic data was used. The main contributions are a documented CD4051–ESP8266 interface, end-to-end integration of five environmental variables with local LCD and cloud data storage, a React/Flask stack with optional Supabase storage, and an ML pipeline validated on real sensor data. The prototype is suitable for lab, greenhouse, or other indoor environments and supports multiple analog sensors read through a single ADC. Reference implementation code is available at github.com/Prateeeek.

ACKNOWLEDGMENT

The authors would like to thank the School of Electronics Engineering, Vellore Institute of Technology, for the support and resources provided throughout this project.

REFERENCES

- [1] S. Kumar and A. Sharma, "Cloud-based air quality monitoring using low-cost IoT sensors," *IEEE Access*, vol. 7, pp. 108602–108612, 2019.
- [2] M. Gupta and R. Das, "IoT-enabled climate control system using NodeMCU," *International Journal of Smart Systems*, vol. 5, no. 3, pp. 115–122, 2021.
- [3] D. Verma and K. Singh, "Machine learning-based anomaly detection in IoT sensor data," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 8, pp. 5678–5686, 2022.
- [4] J. Singh and T. Reddy, "Gas detection and alerting system using MQ sensors and ESP8266," *Sensors and Applications*, vol. 10, no. 2, pp. 32–38, 2023.
- [5] P. Rao and S. Mehta, "Data acquisition through ESP8266 for environmental analytics," *IoT Journal*, vol. 4, no. 1, pp. 1–7, 2022.
- [6] K. Chen and L. Zhang, "Configurable IoT sensor networks for precision laboratory environmental monitoring," *IEEE Transactions on Instrumentation and Measurement*, vol. 69, no. 8, pp. 5421–5430, 2020.
- [7] P. Rao and S. Mehta, "Data acquisition through ESP8266 for environmental analytics," *IoT Journal*, vol. 4, no. 1, pp. 1–7, 2022.
- [8] J. Singh and T. Reddy, "Gas detection and alerting system using MQ sensors and ESP8266," *Sensors and Applications*, vol. 10, no. 2, pp. 32–38, 2023.
- [9] L. Wang et al., "Wireless sensor network for environmental monitoring based on Zigbee," *IEEE Sensors Journal*, vol. 18, no. 2, pp. 755–761, 2018.
- [10] L. Wang, X. Li, Y. Zhang, and H. Chen, "Wireless sensor network for environmental monitoring based on Zigbee," *IEEE Sensors Journal*, vol. 18, no. 2, pp. 755–761, 2018.
- [11] F. Ali, M. Saeed, A. Khan, and S. Khan, "LoRaWAN-based scalable environmental monitoring system," *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 4321–4330, 2020.
- [12] A. Carns, "Streamlit: Fast and interactive dashboards for data science," *Journal of Open Source Software*, vol. 5, no. 51, p. 2174, 2020.
- [13] R. Hipp, "SQLite: Lightweight database engine for embedded systems," *Communications of the ACM*, vol. 60, no. 9, pp. 92–99, 2017.
- [14] R. Patel and S. Jain, "Scalable IoT analytics using AWS cloud platform," *IEEE Cloud Computing*, vol. 6, no. 3, pp. 45–53, 2019.
- [15] M. Anderson et al., "Modular environmental monitoring systems for research laboratories: Design and implementation," *Sensors and Actuators A: Physical*, vol. 325, p. 112703, 2021.
- [16] J. Park and S. Kim, "Real-time customizable monitoring systems for chemical reactor safety," *Journal of Process Control*, vol. 115, pp. 45–58, 2022.
- [17] A. Rodriguez, P. Gomez, and L. Fernandez, "Adaptive IoT platforms for industrial process monitoring and control," *IEEE Transactions on Industrial Electronics*, vol. 70, no. 4, pp. 3821–3832, 2023.
- [18] Y. Li and H. Wang, "IoT-based customizable climate control systems for smart greenhouses," *Computers and Electronics in Agriculture*, vol. 165, p. 104961, 2019.
- [19] T. Martinez et al., "Modular sensor networks for precision agriculture in controlled environments," *Agricultural Systems*, vol. 192, p. 103198, 2021.
- [20] R. Thompson and M. Lee, "Scalable and reconfigurable IoT architectures for industrial applications," *IEEE Internet of Things Journal*, vol. 7, no. 9, pp. 8234–8245, 2020.
- [21] S. Kumar, A. Verma, and P. Singh, "Modular sensor fusion frameworks for customizable environmental monitoring," *IEEE Sensors Journal*, vol. 22, no. 12, pp. 11845–11856, 2022.